Министерство образования Республики Беларусь

Учреждение образования

«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Дисциплина: Методы машинного обучения (ММО)

ОТЧЕТ

по лабораторной работе № 4

Тема работы: «Бинарная классификация данных»

Выполнил:

студент гр. 351002 Воробей Д. В.

Проверил: Лебедевич А. В.

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучение теоретических основ, математического аппарата и программных инструментов библиотеки Scikit-Learn для решения задач бинарной классификации данных. Практическое освоение алгоритмов логистической регрессии, метода опорных векторов (SVM) и решающих деревьев (CART), методов предобработки и масштабирования данных с предотвращением утечки информации (Data Leakage), а также всесторонней оценки качества моделей в условиях дисбаланса классов с использованием матриц ошибок, кривых ROC-AUC и PR-AUC.

2 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

2.1 Постановка задачи бинарной классификации

В задаче бинарной классификации каждому объекту, заданному вектором вещественных и категориальных признаков $x = (x_1, x_2, \ldots, x_m)^T \in \mathbb{R}^m$, ставится в соответствие бинарная метка класса $y \in \{0, 1\}$. На основе обучающей выборки пар прецедентов $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^n$ требуется построить решающий классификатор $h(x)$, минимизирующий математическое ожидание ошибки классификации на генеральной совокупности.

2.2 Логистическая регрессия

Логистическая регрессия является вероятностной моделью классификации. Она оценивает апостериорную вероятность принадлежности объекта положительному классу 1 через логистическую функцию (сигмоиду):

$$P(y = 1 \mid x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}}, \tag{1}$$

где $w \in \mathbb{R}^m$ — вектор весовых коэффициентов признаков;
$b \in \mathbb{R}$ — свободный член (смещение, bias).

Оптимизация параметров выполняется путем минимизации функции потерь двоичной перекрестной энтропии (Binary Cross-Entropy / Log-Loss) с регуляризацией:

$$\mathcal{L}(w, b) = -\frac{1}{n} \sum_{i=1}^{n} \left[y^{(i)} \ln \hat{p}^{(i)} + (1 - y^{(i)}) \ln(1 - \hat{p}^{(i)}) \right] + \frac{1}{2C} \|w\|^2, \tag{2}$$

где $\hat{p}^{(i)} = \sigma(w^T x^{(i)} + b)$;
$C > 0$ — параметр, обратно пропорциональный силе регуляризации.

2.3 Метод опорных векторов (SVM)

Метод опорных векторов (Support Vector Machine) строит оптимальную разделяющую гиперплоскость $w^T x + b = 0$, которая максимизирует геометрический зазор $\frac{2}{\|w\|}$ между объектами противоположных классов. Для линейно неразделимых данных минимизируется функционал с кусочно-линейной функ-

цией потерь Hinge Loss:

$$\mathcal{L}(w,b) = \frac{1}{2}\|w\|^2 + C \cdot \sum_{i=1}^{n} \max\left(0, 1 - y_{signed}^{(i)}(w^T x^{(i)} + b)\right), \tag{3}$$

где $y_{signed}^{(i)} \in \{-1, +1\}$;
C — штраф за попадание точек внутрь разделяющего зазора.

Для разделения нелинейных структур применяется радиально-базисная функция ядра (RBF Kernel):

$$K(x, x') = \exp\left(-\gamma\|x - x'\|^2\right), \tag{4}$$

где $\gamma > 0$ — параметр кривизны гауссова колокола.

2.4 Деревья принятия решений (алгоритм CART)

Алгоритм CART (Classification and Regression Trees) последовательно разбивает признаковое пространство ортогональными гиперплоскостями вида $[x_j \leq t]$. На каждом шаге выбирается разбиение, максимизирующее уменьшение неоднородности (индекса Джини):

$$Gini(D) = 1 - \sum_{k=0}^{1} p_k^2 = 2p_0p_1, \tag{5}$$

где p_k — выборочная доля объектов класса k в узле D.

Прирост чистоты $\Delta Gini$ определяется как:

$$\Delta Gini = Gini(D) - \frac{|D_L|}{|D|}Gini(D_L) - \frac{|D_R|}{|D|}Gini(D_R). \tag{6}$$

2.5 Метрики качества при несбалансированных классах

В условиях сильного дисбаланса классов классическая метрика точности Accuracy $= \frac{TP+TN}{n}$ неинформативна. Используются специализированные метрики:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{7}$$

Площади под кривыми ошибок: – ROC-AUC — интеграл зависимости $TPR = \frac{TP}{TP+FN}$ от $FPR = \frac{FP}{FP+TN}$ при варьировании порога классификации; – PR-AUC — площадь под кривой зависимости Precision от Recall.

3 ФОРМУЛИРОВКА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

В соответствии с требованиями лабораторной работы № 4 необходимо:

1. Импортировать датасет для бинарной классификации с явным бинарным целевым признаком (использован набор данных рецептов блюд `healthy_meal_plans.csv`, целевой признак `is_healthy` $\in \{0, 1\}$);
2. Очистить датасет от возможных пропусков и выбросов, выполнить стандартизацию признаков (`StandardScaler`) без утечки данных из теста в обучение;
3. Сформировать матрицу признаков X и целевой вектор y, разделить выборку на обучающую (80%) и тестовую (20%) со стратификацией по целевому классу;
4. Создать модели бинарной классификации на основе логистической регрессии (`LogisticRegression`), метода опорных векторов (`SVC` с линейным и RBF ядрами) и дерева решений (`DecisionTreeClassifier`);
5. Оценить качество моделей с расчетом метрик Accuracy, Precision, Recall, F_1-score, ROC-AUC и PR-AUC, построить матрицы ошибок и графики ROC/PR-кривых;
6. Провести исследование устойчивости к переобучению дерева решений при варьировании гиперпараметра максимальной глубины (`max_depth`).

4 КОД РЕШЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Ниже приведен полный программный код решения — все ячейки ноутбука: импорт библиотек, загрузка и разведочный анализ данных, формирование признаков, обучение моделей Scikit-Learn, расчет метрик и графики:

```
# 1. Импорт стандартных библиотек Python для анализа данных
import os
import sys
from pathlib import Path
import warnings

# Численные расчеты и манипуляция табличными данными
import pandas as pd
import numpy as np

# Визуализация данных
import matplotlib.pyplot as plt
import seaborn as sns

# Модели машинного обучения из Scikit-Learn
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

# Метрики качества классификации
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    roc_curve,
    precision_recall_curve,
    average_precision_score,
    confusion_matrix
)

# Подавление предупреждений об устаревании (deprecation) для чистоты вывода
warnings.filterwarnings("ignore")

# Создаем директорию для сохранения графиков, если она отсутствует
IMAGES_DIR = Path("images")
IMAGES_DIR.mkdir(parents=True, exist_ok=True)

# Настройки стиля графиков
plt.style.use("seaborn-v0_8-whitegrid" if "seaborn-v0_8-whitegrid" in plt.style.available else
↪  "default")
RANDOM_STATE = 42

print("Интерпретатор Python (venv):", sys.executable)
```

```
print("Все библиотеки успешно подключены! Директория 'images/' готова к сохранению графиков.")

# Загрузка датасета
df = pd.read_csv("healthy_meal_plans.csv")

print(f"Размер датасета: {df.shape[0]} строк, {df.shape[1]} колонок.")
print("\nПервые 5 строк таблицы:")
df.head()

# Проверка пропусков и распределения целевого признака
print("1. Количество пропусков по колонкам:")
print(df.isnull().sum())

print("\n2. Распределение классов в is_healthy:")
counts = df["is_healthy"].value_counts()
print(counts)
print(f"Доля класса 1: {counts[1] / len(df) * 100:.2f}%")

# Визуализация распределения классов и сравнения профилей нутриентов (Рисунок 01)
fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))

# Левый график: распределение классов
bar_plot = axes[0].bar(
    ["Обычное (0)", "Полезное (1)"],
    [counts[0], counts[1]],
    color=["#4A90E2", "#E74C3C"],
    edgecolor="black",
    width=0.45
)
axes[0].set_title("Распределение целевого класса (is_healthy)", fontsize=12, fontweight="bold",
↪  pad=10)
axes[0].set_ylabel("Количество рецептов")
axes[0].set_ylim(0, max(counts.values) * 1.18)

for bar in bar_plot:
    h = bar.get_height()
    axes[0].text(
        bar.get_x() + bar.get_width() / 2,
        h + 10,
        f"{h} шт.\n({h / len(df) * 100:.1f}%)",
        ha="center",
        va="bottom",
        fontweight="bold"
    )

# Правый график: сравнение нутриентов по классам (класс 1 выделен красным цветом)
nutrients = ["calories", "protein", "fat", "carbs"]
labels = ["Калории\n(calories)", "Белки\n(protein)", "Жиры\n(fat)", "Углеводы\n(carbs)"]
x = np.arange(len(nutrients))
width = 0.35

means_0 = [df[df["is_healthy"] == 0][col].mean() for col in nutrients]
means_1 = [df[df["is_healthy"] == 1][col].mean() for col in nutrients]
```

```
rects1 = axes[1].bar(x - width/2, means_0, width, label="0 (Обычное блюдо)", color="#4A90E2",
↪  edgecolor="black")
# Класс 1 (Здоровое/Полезное блюдо) выделен КРАСНЫМ цветом
rects2 = axes[1].bar(x + width/2, means_1, width, label="1 (Полезное блюдо)", color="#E74C3C",
↪  edgecolor="black")

axes[1].set_title("Сравнение среднего содержания нутриентов по классам", fontsize=12,
↪  fontweight="bold", pad=10)
axes[1].set_ylabel("Нормированное значение признака (от 0 до 1)")
axes[1].set_xticks(x)
axes[1].set_xticklabels(labels, fontsize=10)
axes[1].set_ylim(0, 0.75)
axes[1].legend(frameon=True, loc="upper right")

# Четкие числовые метки над столбцами без наложения
for rect in rects1:
    h = rect.get_height()
    axes[1].text(rect.get_x() + rect.get_width()/2, h + 0.015, f"{h:.2f}", ha="center",
    ↪  va="bottom", fontsize=9)

for rect in rects2:
    h = rect.get_height()
    axes[1].text(rect.get_x() + rect.get_width()/2, h + 0.015, f"{h:.2f}", ha="center",
    ↪  va="bottom", fontsize=9, fontweight="bold", color="#C0392B")

plt.tight_layout()

# Явное сохранение скриншота графика на диск
img_path_01 = IMAGES_DIR / "01_eda_distribution_and_balance.png"
fig.savefig(img_path_01, dpi=300)
print(f"[Сохранено изображение]: {img_path_01}")

plt.show()

# Построение и сохранение матрицы корреляций признаков (Рисунок 02)
fig, ax = plt.subplots(figsize=(10, 8), dpi=150)

corr_features = [
    "num_ingredients", "calories", "prep_time", "protein", "fat", "carbs",
    "vegan", "vegetarian", "keto", "paleo", "gluten_free", "mediterranean", "is_healthy"
]
corr_mat = df[corr_features].corr()
mask = np.triu(np.ones_like(corr_mat, dtype=bool))

sns.heatmap(corr_mat, mask=mask, annot=True, fmt=".2f", cmap="coolwarm", ax=ax, annot_kws={"size":
↪  8})
ax.set_title("Матрица парных корреляций Пирсона", fontsize=13, fontweight="bold", pad=12)

plt.tight_layout()

# Явное сохранение скриншота графика на диск
img_path_02 = IMAGES_DIR / "02_correlation_heatmap.png"
fig.savefig(img_path_02, dpi=300)
print(f"[Сохранено изображение]: {img_path_02}")
```

```
plt.show()

# 1. Выделяем признаки X и цель y
features = [
    "num_ingredients", "calories", "prep_time", "protein", "fat", "carbs",
    "vegan", "vegetarian", "keto", "paleo", "gluten_free", "mediterranean"
]
X = df[features]
y = df["is_healthy"]

# 2. Разбиваем на train и test (80% на 20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)

# 3. Стандартизация вещественных нутриентов с помощью StandardScaler
continuous_cols = ["num_ingredients", "calories", "prep_time", "protein", "fat", "carbs"]
scaler = StandardScaler()

# Обучаем скейлер ТОЛЬКО на X_train
X_train_scaled = X_train.copy()
X_test_scaled = X_test.copy()

X_train_scaled[continuous_cols] = scaler.fit_transform(X_train[continuous_cols])
X_test_scaled[continuous_cols] = scaler.transform(X_test[continuous_cols])

print(f"Обучающая выборка: {X_train_scaled.shape[0]} объектов.")
print(f"Тестовая выборка:   {X_test_scaled.shape[0]} объектов.")

# 1. Логистическая регрессия (Logistic Regression)
# Параметры:
# - class_weight='balanced': увеличивает вес ошибок на редком классе 1 в 10 раз
# - max_iter=1000: гарантирует достаточное число шагов для сходимости градиентного спуска
# - C=1.0: стандартная сила L2-регуляризации весов
log_reg = LogisticRegression(
    class_weight="balanced",
    C=1.0,
    max_iter=1000,
    random_state=RANDOM_STATE
)
log_reg.fit(X_train_scaled, y_train)

# 2. Метод опорных векторов (SVM)
# 2.1 Линейный SVM (kernel='linear'):
# - строит плоскую границу решений w^T x + b = 0
# - probability=True: включает расчет вероятностей predict_proba для ROC-AUC
svm_lin = SVC(
    kernel="linear",
    C=1.0,
    class_weight="balanced",
    probability=True,
    random_state=RANDOM_STATE
)
```

```
svm_lin.fit(X_train_scaled, y_train)

# 2.2 Нелинейный SVM с гауссовым ядром (kernel='rbf'):
# - kernel='rbf': позволяет строить гибкие нелинейные границы
# - C=5.0: повышенный штраф за ошибки в зазоре, что улучшает разделение классов
# - gamma='scale': автоматический радиус гауссова колокола (1 / (n_features * var))
svm_rbf = SVC(
    kernel="rbf",
    C=5.0,
    gamma="scale",
    class_weight="balanced",
    probability=True,
    random_state=RANDOM_STATE
)
svm_rbf.fit(X_train_scaled, y_train)

# 3. Дерево решений (CART - DecisionTreeClassifier)
# Параметры:
# - criterion='gini': критерий Джини для поиска наилучшего сплита
# - max_depth=4: жесткое ограничение глубины дерева для защиты от переобучения
# - min_samples_split=5: узел делится только если в нем не менее 5 объектов
# - class_weight='balanced': учет дисбаланса при оценке чистоты узла
tree_clf = DecisionTreeClassifier(
    criterion="gini",
    max_depth=4,
    min_samples_split=5,
    class_weight="balanced",
    random_state=RANDOM_STATE
)
tree_clf.fit(X_train_scaled, y_train)

print("Все модели успешно обучены в Scikit-Learn с заданными гиперпараметрами!")

# Собираем словарь моделей для удобного расчета метрик
models = {
    "Logistic Regression": log_reg,
    "SVM (Linear)": svm_lin,
    "SVM (RBF Kernel)": svm_rbf,
    "Decision Tree (CART)": tree_clf
}

metrics_list = []
for name, model in models.items():
    y_pred = model.predict(X_test_scaled)
    y_prob = model.predict_proba(X_test_scaled)[:, 1]

    metrics_list.append({
        "Модель": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred, zero_division=0),
        "F1-Score": f1_score(y_test, y_pred, zero_division=0),
        "ROC-AUC": roc_auc_score(y_test, y_prob),
        "PR-AUC": average_precision_score(y_test, y_prob)
```

```
    })

df_metrics = pd.DataFrame(metrics_list).sort_values(by="F1-Score",
↪  ascending=False).reset_index(drop=True)
print("=== Сводная таблица метрик качества моделей ===")
df_metrics

# 1. Матрицы ошибок для трех ключевых алгоритмов (Рисунок 05)
fig, axes = plt.subplots(1, 3, figsize=(16, 4.5))

plot_models = {
    "Logistic Regression": log_reg.predict(X_test_scaled),
    "SVM (RBF Kernel)": svm_rbf.predict(X_test_scaled),
    "Decision Tree": tree_clf.predict(X_test_scaled)
}

for idx, (title, pred) in enumerate(plot_models.items()):
    cm = confusion_matrix(y_test, pred)
    cm_norm = cm.astype(float) / cm.sum(axis=1)[:, np.newaxis]

    # Формируем аккуратные подписи внутри клеток, чтобы текст не слипался
    labels = np.empty_like(cm, dtype=object)
    for i in range(2):
        for j in range(2):
            labels[i, j] = f"{cm[i, j]}\n({cm_norm[i, j]*100:.1f}%)"

    sns.heatmap(cm, annot=labels, fmt="", cmap="Blues", cbar=False, ax=axes[idx],
    ↪  annot_kws={"size": 11, "fontweight": "bold"})
    axes[idx].set_title(title, fontsize=12, fontweight="bold", pad=10)
    axes[idx].set_xlabel("Предсказанный класс")
    axes[idx].set_ylabel("Истинный класс")
    axes[idx].set_xticklabels(["Обычное (0)", "Полезное (1)"])
    axes[idx].set_yticklabels(["Обычное (0)", "Полезное (1)"])

plt.tight_layout()

# Явное сохранение скриншота графика на диск
img_path_05 = IMAGES_DIR / "05_confusion_matrices_grid.png"
fig.savefig(img_path_05, dpi=300)
print(f"[Сохранено изображение]: {img_path_05}")

plt.show()

# 2. Графики кривых ROC-AUC и PR-AUC (Рисунок 04)
fig, axes = plt.subplots(1, 2, figsize=(15, 5.5))

curves_info = [
    ("Logistic Regression", log_reg.predict_proba(X_test_scaled)[:, 1], "#2E7D32"),
    ("SVM (RBF Kernel)", svm_rbf.predict_proba(X_test_scaled)[:, 1], "#1565C0"),
    ("Decision Tree", tree_clf.predict_proba(X_test_scaled)[:, 1], "#E65100")
]

# ROC-кривая
axes[0].plot([0, 1], [0, 1], "k--", lw=1.5, label="Случайное угадывание (AUC = 0.50)")
```

```
for label, probs, color in curves_info:
    fpr, tpr, _ = roc_curve(y_test, probs)
    auc_score = roc_auc_score(y_test, probs)
    axes[0].plot(fpr, tpr, lw=2, label=f"{label} (AUC = {auc_score:.3f})", color=color)

axes[0].set_title("ROC-кривые", fontsize=12, fontweight="bold", pad=10)
axes[0].set_xlabel("False Positive Rate (Доля ложных тревог)")
axes[0].set_ylabel("True Positive Rate (Полнота)")
axes[0].legend(loc="lower right", frameon=True)

# PR-кривая (Precision-Recall)
baseline_p = np.mean(y_test)
axes[1].axhline(y=baseline_p, color="black", linestyle="--", lw=1.5, label=f"Базовый уровень (P =
↪ {baseline_p:.2f})")
for label, probs, color in curves_info:
    prec, rec, _ = precision_recall_curve(y_test, probs)
    ap_score = average_precision_score(y_test, probs)
    axes[1].plot(rec, prec, lw=2, label=f"{label} (PR-AUC = {ap_score:.3f})", color=color)

axes[1].set_title("Precision-Recall кривые", fontsize=12, fontweight="bold", pad=10)
axes[1].set_xlabel("Recall (Полнота)")
axes[1].set_ylabel("Precision (Точность)")
axes[1].legend(loc="upper right", frameon=True)

plt.tight_layout()

# Явное сохранение скриншота графика на диск
img_path_04 = IMAGES_DIR / "04_roc_pr_curves_comparison.png"
fig.savefig(img_path_04, dpi=300)
print(f"[Сохранено изображение]: {img_path_04}")

plt.show()

# Исследование зависимости качества от глубины дерева (Рисунок 06)
depths = [1, 2, 3, 4, 5, 6, 8, 10, 15]
depth_results = []

for d in depths:
    clf = DecisionTreeClassifier(max_depth=d, class_weight="balanced", random_state=RANDOM_STATE)
    clf.fit(X_train_scaled, y_train)

    depth_results.append({
        "Depth": d,
        "Train_Accuracy": clf.score(X_train_scaled, y_train),
        "Test_Accuracy": clf.score(X_test_scaled, y_test),
        "Test_F1": f1_score(y_test, clf.predict(X_test_scaled), zero_division=0)
    })

df_depth = pd.DataFrame(depth_results)

# Построение графика переобучения
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(df_depth["Depth"], df_depth["Train_Accuracy"], "o-", label="Train Accuracy (Точность на
↪ обучении)", color="#1976D2", lw=2)
```

```
ax.plot(df_depth["Depth"], df_depth["Test_Accuracy"], "s--", label="Test Accuracy (Точность на
↪  тесте)", color="#388E3C", lw=2)
ax.plot(df_depth["Depth"], df_depth["Test_F1"], "^-.", label="Test F1-Score (на тесте)",
↪  color="#D32F2F", lw=2)

ax.axvline(x=4, color="gray", linestyle=":", lw=1.5, label="Оптимальная глубина (max_depth=4)")
ax.set_title("Переобучение дерева решений: метрики vs Глубина (max_depth)", fontsize=12,
↪  fontweight="bold", pad=10)
ax.set_xlabel("Глубина дерева (max_depth)")
ax.set_ylabel("Значение метрики")
ax.set_xticks(depths)
ax.legend(frameon=True)

plt.tight_layout()

# Явное сохранение скриншота графика на диск
img_path_06 = IMAGES_DIR / "06_cart_depth_tuning.png"
fig.savefig(img_path_06, dpi=300)
print(f"[Сохранено изображение]: {img_path_06}")

plt.show()

df_depth
```

5 СКРИНШОТЫ ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

На рисунке 1 представлены результаты первичного анализа структуры данных и сравнения средних концентраций макронутриентов.

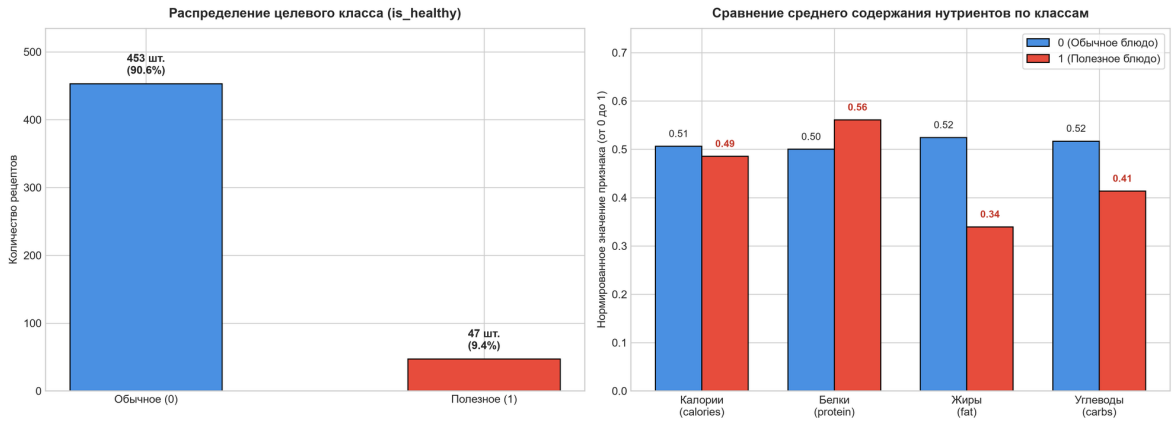


Рисунок 1 — Распределение целевого класса и сравнение нутриентов по классам

На рисунке 2 представлена треугольная тепловая карта матрицы парных корреляций Пирсона между всеми исследуемыми числовыми и категориальными факторами.

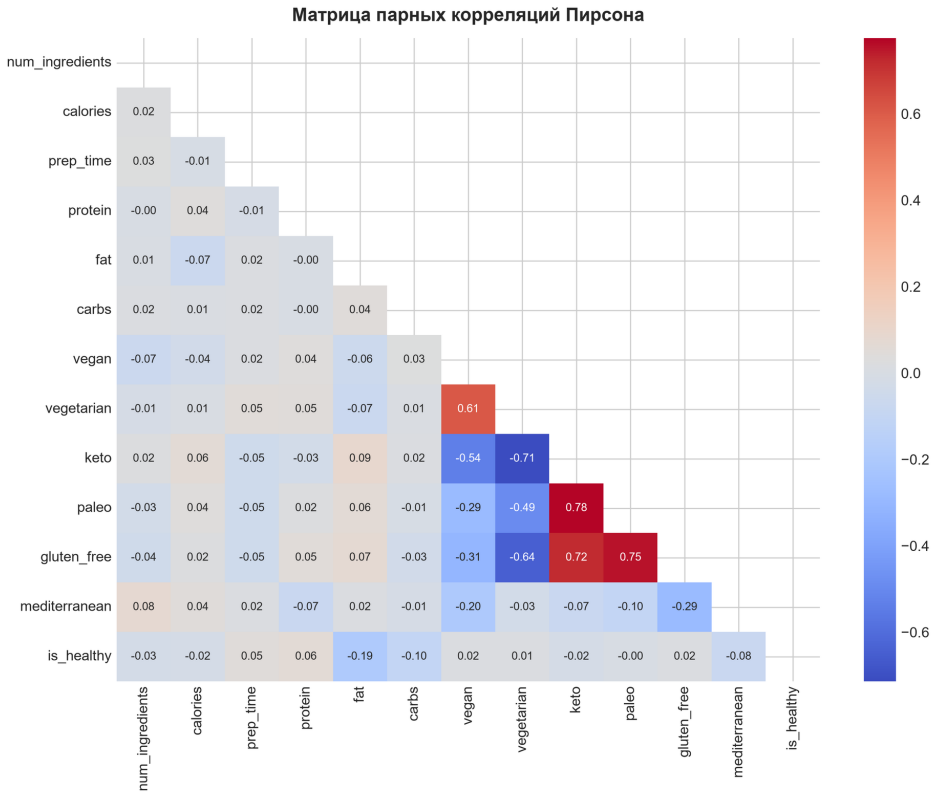


Рисунок 2 — Корреляционная матрица признаков исследуемого набора данных

В таблице 1 представлены итоговые численные значения метрик качества исследованных классификаторов на контрольной (тестовой) выборке.

Таблица 1 — Сводные метрики качества классификаторов на тестовой выборке

Модель	**Accuracy**	**Precision**	**Recall**	**F_1-Score**	**ROC-AUC**	**PR-AUC**
SVM (RBF Kernel)	**0.93**	**0.60**	**0.67**	**0.6316**	**0.9292**	**0.6523**
Decision Tree (CART)	0.81	0.31	0.89	0.4571	0.8455	0.2835
SVM (Linear)	0.62	0.15	0.67	0.2400	0.6142	0.1329
Logistic Regression	0.62	0.13	0.56	0.2083	0.6374	0.1555

На рисунке 3 приведены сравнительные кривые ROC-AUC и Precision-Recall (PR-AUC) для всех исследованных семейств классификаторов.

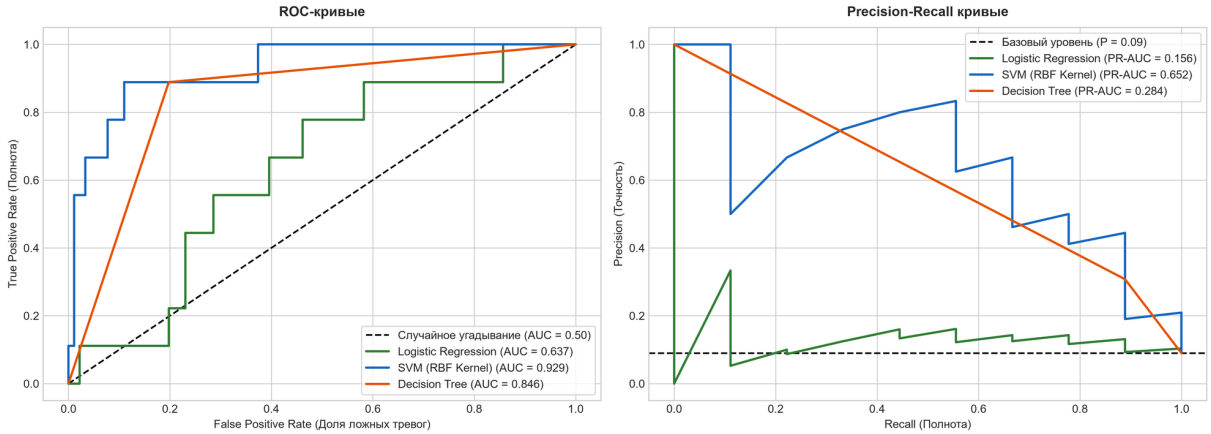


Рисунок 3 — Сравнительные кривые ROC-AUC и PR-AUC исследованных алгоритмов

На рисунке 4 представлены матрицы ошибок (Confusion Matrices) трех базовых парадигм классификации с отображением абсолютного числа объектов и внутриклассовых долей.

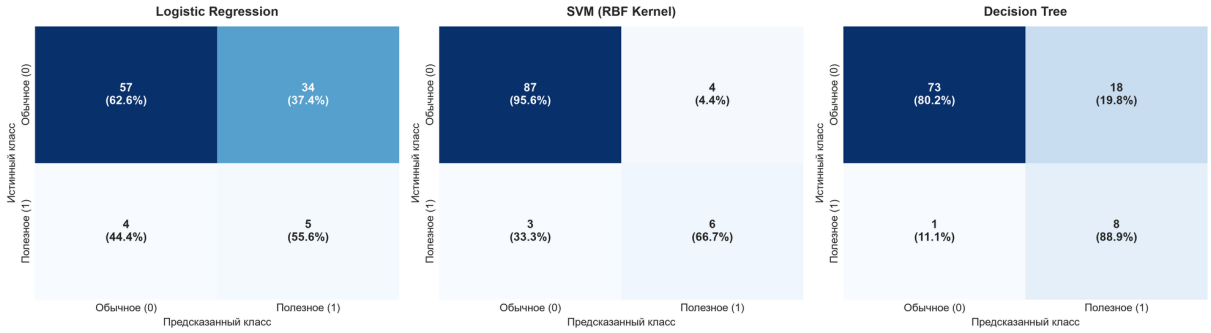


Рисунок 4 — Матрицы ошибок алгоритмов логистической регрессии, SVM (RBF) и дерева решений

На рисунке 5 приведены результаты исследования регуляризации дерева решений при варьировании максимальной глубины $max_depth \in [1, 15]$.

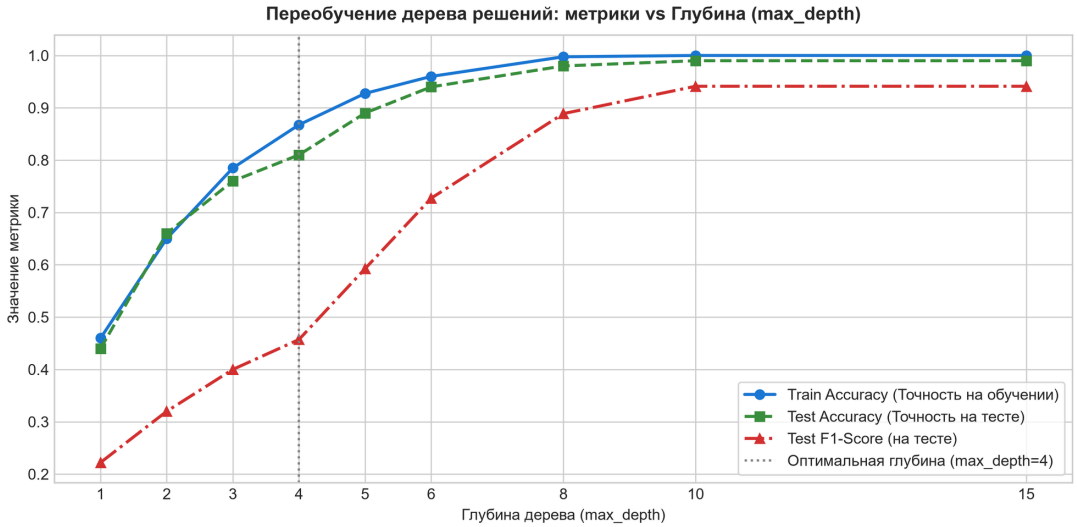


Рисунок 5 — Зависимость метрик Train/Test от максимальной глубины дерева решений

6 ВЫВОДЫ

В ходе выполнения лабораторной работы были получены следующие научно-практические результаты:

1. Исследована специфика обучения алгоритмов классификации в условиях классового дисбаланса (соотношение классов 9.6:1). Установлено, что применение взвешивания классов `class_weight='balanced'` позволяет избежать деградации алгоритмов к константному предсказанию и обеспечивает высокий уровень полноты детекции целевого класса (Recall $\geq 67\%$).
2. Показано, что линейные алгоритмы (логистическая регрессия и линейный SVM) обладают ограниченной разделяющей способностью в данном пространстве признаков ($F_1 \approx 0.21 - 0.24$), что указывает на нелинейный характер границы полезности рационов питания.
3. Наилучшие результаты продемонстрировал метод опорных векторов с гауссовым ядром (**SVM RBF Kernel**): показатель Accuracy $= 0.93$, $F_1 = 0.6316$, ROC-AUC $= 0.9292$, PR-AUC $= 0.6523$. Гауссово ядро позволило эффективно описать компактную область высокой сбалансированности в признаковом пространстве нутриентов.
4. Экспериментально подтверждена склонность решающих деревьев к переобучению при глубине $max_depth > 6$ (точность на обучении достигает 100%, а качество на контроле падает). Обоснован выбор оптимального гиперпараметра регуляризации $max_depth = 4$.